

Proyecto de Software

Bases de datos: persistir y cuidar las solicitudes

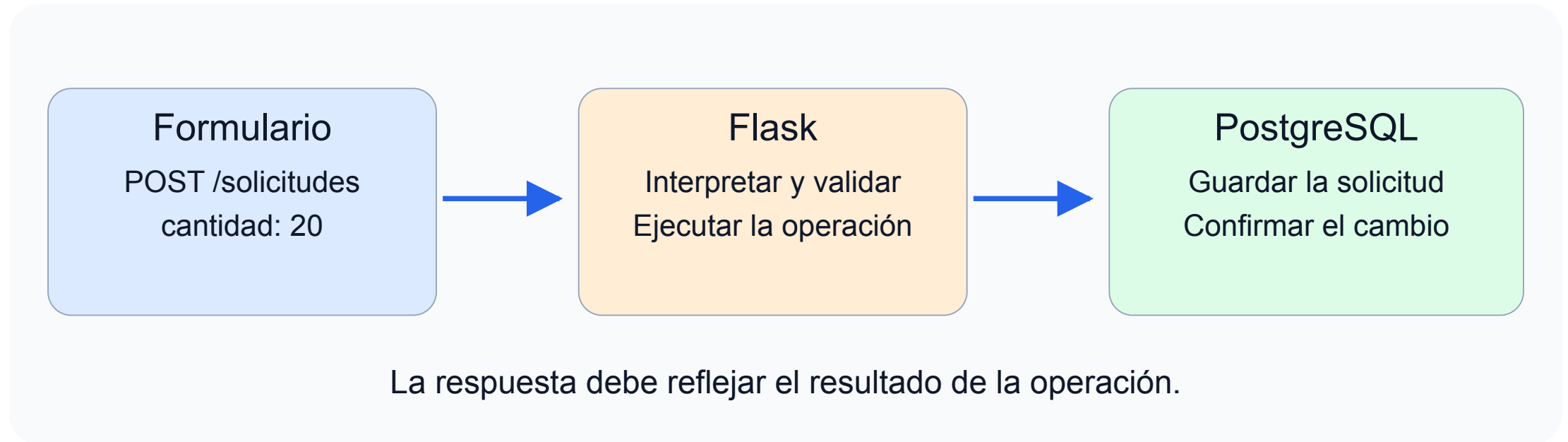
Recorrido de la clase

1. Persistencia y modelo relacional.
2. SQL: leer y modificar datos.
3. Integridad y transacciones.
4. Acceso desde Python.
5. Revisar lo que genera un agente.

No buscamos memorizar SQL: buscamos explicar qué datos cambia una operación y qué garantías necesitamos.

Del formulario a los datos

En Flask recibimos una cantidad mediante un formulario.



Una respuesta exitosa no demuestra que los datos estén persistidos.

Una lista no es una base de datos

```
solicitudes = []  
solicitudes.append({"id": 1, "cantidad": 20})
```

- Al reiniciar el proceso, perdemos los cambios.
- Distintos procesos pueden tener listas diferentes.
- No tenemos las garantías de integridad de una base de datos.

La memoria sirve para una demostración; no para registrar entregas reales.

Nuestro ejemplo, deliberadamente pequeño

Trabajaremos con **servicios** y **solicitudes de comidas**.

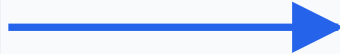
Una solicitud tendrá un servicio, una fecha y una cantidad pedida.

Simplificación didáctica: no modelamos personas, dietas, permisos, autorizaciones ni entregas.

El objetivo es aprender SQL, no diseñar un sistema completo.

Modelo relacional

servicios	
id (PK)	nombre
1	Servicio A
2	Servicio B



solicitudes			
id (PK)	servicio_id (FK)	fecha	cantidad
10	1	2026-09-08	20
11	1	2026-09-09	15

PK: identifica una fila. FK: referencia una fila de otra tabla.

- **Filas:** registros. **Columnas:** atributos con significado y tipo.
- **Claves:** identifican registros y conectan las tablas.

No son lo mismo

Elemento	Rol
PostgreSQL	Motor que administra datos y ejecuta consultas
SQL	Lenguaje para definir, consultar y modificar datos
psycopg2	Adaptador para acceder a PostgreSQL desde Python
pgAdmin / psql	Cientes para trabajar con el motor
ORM	Otra forma de expresar parte del acceso desde objetos

Cambiar de cliente no cambia la base de datos.

pgAdmin: una interfaz, no el motor



pgAdmin permite conectarse al motor y trabajar con las bases de datos.

El cliente envía consultas; **PostgreSQL** administra los datos.

Definir servicios

```
CREATE TABLE servicios (  
    id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    nombre text NOT NULL UNIQUE  
);
```

- **PRIMARY KEY** : identidad única y no nula.
- **IDENTITY** : el motor genera el identificador.
- **NOT NULL** : el dato no puede faltar.
- **UNIQUE** : no se repite el nombre en este ejemplo.

Estas reglas son decisiones, no decoración del esquema.

Definir solicitudes

```
CREATE TABLE solicitudes (  
    id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    servicio_id integer NOT NULL REFERENCES servicios(id),  
    fecha date NOT NULL,  
    cantidad integer NOT NULL CHECK (cantidad > 0)  
);
```

La clave foránea exige que el servicio exista.

Regla del ejemplo: la cantidad pedida debe ser positiva.

Checkpoint: ¿qué rechaza la base?

Supongamos que sólo existe el servicio con `id = 1`.

Solicitud	servicio_id	cantidad
A	1	20
B	99	20
C	1	-2
D	1	NULL

Todas tienen una fecha válida.

¿Qué restricción interviene en cada caso?

Insertar sin inventar identificadores

```
INSERT INTO servicios (nombre)
VALUES ('Servicio A')
RETURNING id;

-- Usar el id devuelto; aquí suponemos que fue 1.
INSERT INTO solicitudes (servicio_id, fecha, cantidad)
VALUES (1, DATE '2026-09-08', 20)
RETURNING id;
```

No calcular `MAX(id) + 1`: dos operaciones podrían elegir el mismo valor.

Los identificadores no tienen por qué ser consecutivos.

Leer lo que necesitamos

```
SELECT id, fecha, cantidad  
FROM solicitudes  
WHERE servicio_id = 1  
ORDER BY fecha DESC, id DESC;
```

- `SELECT` : qué columnas necesitamos.
- `WHERE` : qué filas buscamos.
- `ORDER BY` : qué orden esperamos.

Sin `ORDER BY` , no debemos depender del orden de las filas.

Relacionar datos con JOIN

```
SELECT s.id, se.nombre, s.fecha, s.cantidad  
FROM solicitudes AS s  
JOIN servicios AS se ON se.id = s.servicio_id  
ORDER BY s.id;
```

La solicitud guarda la referencia, no una copia del nombre del servicio.

¿Qué pasaría si copiáramos ese nombre en cientos de solicitudes y luego cambiara?

Resumir datos

```
SELECT servicio_id, SUM(cantidad) AS total_pedido  
FROM solicitudes  
WHERE fecha = DATE '2026-09-08'  
GROUP BY servicio_id;
```

Este resultado suma **cantidades pedidas**.

No informa cuánto se autorizó ni cuánto se entregó.

Una consulta válida puede responder la pregunta equivocada.

Modificar con alcance explícito

```
UPDATE solicitudes  
SET cantidad = 25  
WHERE id = 10  
RETURNING id, cantidad;
```

Antes de ejecutar, preguntar:

- ¿Qué filas puede modificar?
- ¿Está permitido cambiar ese dato en este momento?
- ¿Qué significa que no devuelva ninguna fila?

En una aplicación también importan el estado, los permisos y la historia.

Checkpoint: SQL generado por un agente

Pedido: "Corregí a 25 la cantidad de la solicitud 10".

```
UPDATE solicitudes  
SET cantidad = 25;
```

1. ¿Tiene un error de sintaxis?
2. ¿Qué filas modifica?
3. ¿Qué revisaríamos antes de ejecutarlo?

Probar sobre una copia de datos no reemplaza revisar el alcance.

Borrar también es una decisión

```
DELETE FROM solicitudes  
WHERE id = 10;
```

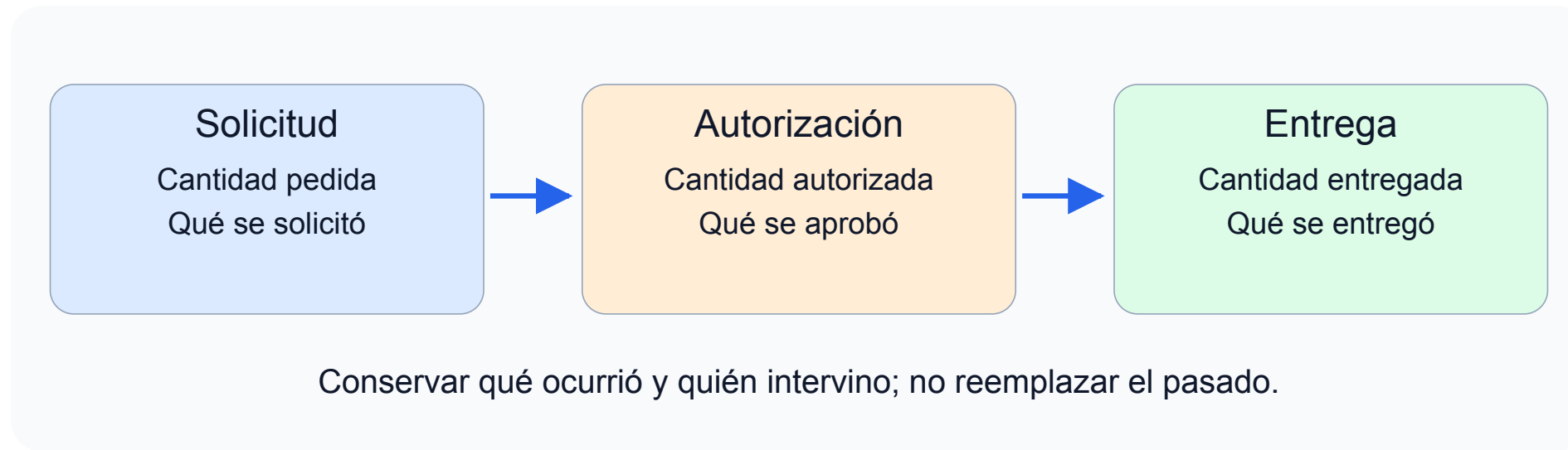
Que SQL permita borrar no significa que el negocio lo permita.

- ¿Hay registros relacionados?
- ¿Necesitamos conservar trazabilidad?
- ¿Corresponde cancelar en vez de borrar?

No ejecutar ejemplos destructivos sobre datos reales.

La historia no debe desaparecer

En un circuito de pedidos podemos distinguir:



Reemplazar siempre un único campo `cantidad` puede borrar información.

Nuestro esquema reducido enseña SQL; **no resuelve esa historia.**

El diseño real debe permitir explicar qué ocurrió y quién intervino.

¿Validar en Python o en la base?

En la aplicación: interpretar entradas, dar mensajes útiles y aplicar reglas del caso de uso.

En la base: proteger invariantes mediante tipos y restricciones, también frente a otros clientes.

Son defensas complementarias.

Un formulario con `min="1"` no impide enviar un POST manual con `cantidad=-2`.

Una operación, varios cambios

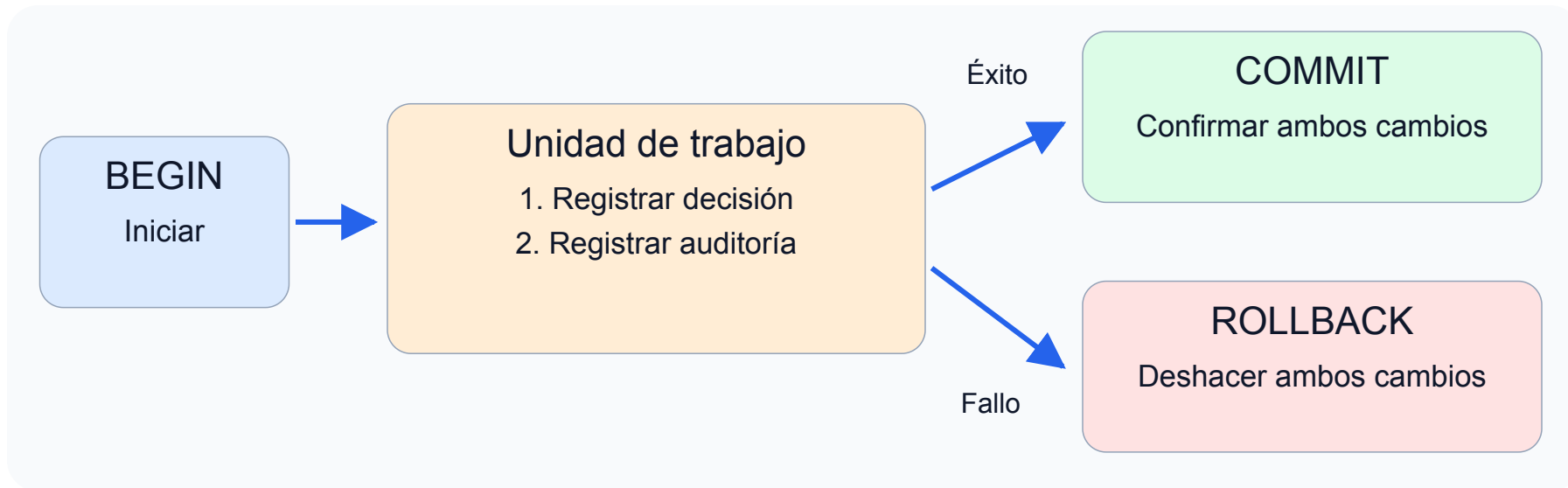
Imaginemos una autorización que debe:

1. registrar la decisión;
2. registrar su responsable en una auditoría.

¿Qué pasa si se guarda el primer cambio y falla el segundo?

Necesitamos definir una **unidad de trabajo**.

Transacciones: todo o nada



Si no podemos completar la operación: **ROLLBACK** .

Los cambios se confirman como una unidad o se deshacen.

La frontera de la transacción debe corresponder a la operación, no a cada línea de SQL.

Una transacción no resuelve todo

Dos personas leen “pendiente” e intentan autorizar al mismo tiempo.

Además de atomicidad necesitamos pensar en **concurrency**:

- restricciones;
- actualizaciones condicionadas;
- aislamiento y bloqueos, según el caso.

No implementaremos hoy ese circuito. Debemos reconocer el riesgo y no suponer que `commit()` lo elimina.

Acceso desde Python

Para acceder a PostgreSQL desde Python usamos **psycopg2**.

En un proyecto de demostración administrado con Poetry:

```
poetry add psycopg2-binary
```

El paquete binario facilita el entorno didáctico; el despliegue requiere revisar sus condiciones de instalación.

Necesitamos PostgreSQL disponible y una base de prueba preparada.

Configuración fuera del código

```
import os
from contextlib import closing

import psycopg2
from psycopg2.extras import RealDictCursor

# DSN configurado en el entorno, sin guardarlo en Git.
dsn = os.environ["DATABASE_URL"]
```

No mostrar credenciales en capturas, prompts ni mensajes de error.

Usar la configuración del entorno de trabajo.

Consultar con parámetros

Con los imports y `dsn` anteriores:

```
with closing(psycopg2.connect(dsn)) as conn:
    with conn:
        with conn.cursor(cursor_factory=RealDictCursor) as cur:
            cur.execute(
                "SELECT id, fecha, cantidad FROM solicitudes "
                "WHERE servicio_id = %s ORDER BY id",
                (servicio_id, ),
            )
            filas = cur.fetchall()
```

`servicio_id` es un entero ya validado por la aplicación.

Valores separados del SQL

```
cur.execute(  
    "SELECT id FROM solicitudes WHERE servicio_id = %s",  
    (servicio_id, ),  
)
```

- `%s` es un marcador del adaptador, no interpolación de Python.
- Los valores viajan en el segundo argumento.
- No poner comillas alrededor de `%s`.
- La coma forma una tupla de un elemento.

Los parámetros son para **valores**, no para nombres de tablas o columnas.

Checkpoint: dos consultas parecidas

A — No usar

```
cur.execute(f"SELECT id FROM solicitudes WHERE servicio_id = {dato}")
```

B

```
cur.execute("SELECT id FROM solicitudes WHERE servicio_id = %s", (dato,))
```

¿Cuál mantiene los datos separados de la estructura SQL?

Parametrizar no reemplaza validar ni verificar permisos.

Insertar y recuperar el identificador

Dentro de los mismos contextos de conexión y cursor:

```
cur.execute(
    "INSERT INTO solicitudes (servicio_id, fecha, cantidad) "
    "VALUES (%s, %s, %s) RETURNING id",
    (servicio_id, fecha, cantidad),
)
solicitud_id = cur.fetchone()["id"]
```

Los tres valores ya fueron interpretados y validados.

La confirmación ocurre al salir exitosamente de `with conn`.

¿Qué cierran esos contextos?

En **psycopg2**:

- `with conn` : confirma al salir sin error; revierte ante una excepción.
- `with conn.cursor(...)` : cierra el cursor.
- `closing(...)` : cierra la conexión, también ante errores.

`with conn` **por sí solo no cierra la conexión.**

Este ejemplo abre una conexión por operación. El manejo de conexiones en una aplicación mayor requiere otra planificación.

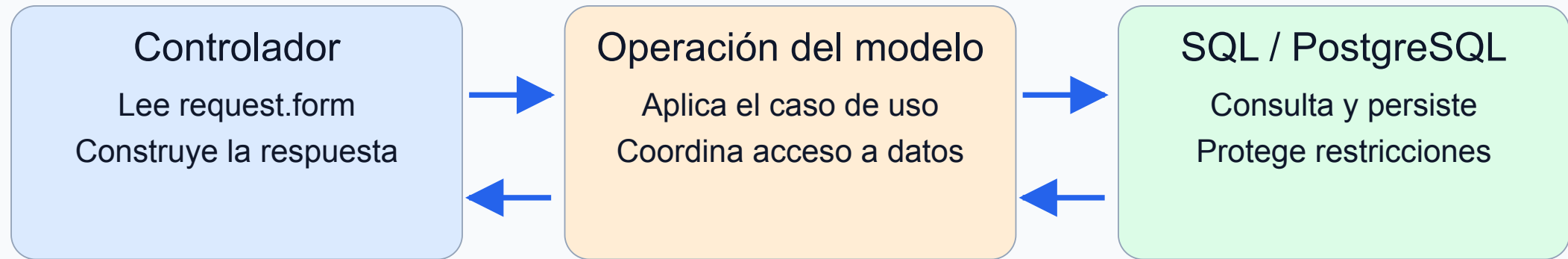
Diagnosticar sin ocultar el problema

Síntoma	Qué revisar
No conecta	Servicio, configuración, red, credenciales
Tabla inexistente	Base seleccionada y esquema preparado
Violación de una restricción	Datos y regla que se incumplió
No aparecen cambios	Transacción, base consultada y filtro

No responder “creada correctamente” después de capturar cualquier excepción.

Registrar el diagnóstico sin exponer secretos; al usuario, un mensaje apropiado.

Persistencia y responsabilidades



El acceso a datos no lee el formulario ni construye HTML.

El acceso a datos no debería construir HTML ni leer `request.form`.

En MVC profundizamos esta separación.

Si cambia la persistencia, queremos limitar qué partes deben modificarse.

Revisar un cambio propuesto por un agente

Antes de aceptar código de acceso a datos:

1. Explicar qué filas lee o modifica.
2. Revisar parámetros, validaciones y permisos.
3. Identificar restricciones y frontera de transacción.
4. Comprobar cierres y manejo de errores.
5. Probar casos válidos, inválidos y fallos.
6. Revisar si conserva la información del negocio.

“No dio error” es una evidencia muy débil.

Evidencias, no sólo explicaciones

Para la creación de una solicitud, pedir pruebas de:

- cantidad válida: se guarda y devuelve su identificador;
- cantidad negativa: se rechaza;
- servicio inexistente: se rechaza;
- fallo a mitad de una unidad de trabajo: no queda incompleta.

Ejecutar en una base descartable, nunca en producción.

Pedir al agente que explique sus pruebas no sustituye ejecutarlas y revisarlas.

¿Qué problema abre la puerta al ORM?

Empiezan a repetirse:

- traducciones entre filas y estructuras de Python;
- operaciones sobre entidades y relaciones;
- código de acceso a datos.

Un ORM ayuda con parte de ese trabajo.

No elimina SQL, restricciones, transacciones ni decisiones de diseño.

Lo importante de hoy

- Persistir no es guardar en una lista.
- El esquema expresa reglas sobre los datos.
- Una consulta correcta debe responder la pregunta correcta.
- Los valores se parametrizan, no se interpolan.
- Las transacciones delimitan unidades de trabajo.
- El equipo responde por el código que acepta de un agente.

Referencias

- [PostgreSQL: tutorial SQL](#)
- [PostgreSQL: restricciones](#)
- [PostgreSQL: transacciones](#)
- [Psycopg2: uso y parámetros](#)
- [Psycopg2: instalación](#)
- [pgAdmin: documentación](#)

Fin

De guardar datos a comprender su significado.